Qiskit v2 Hands-on Introduction

Gwonhak Lee
Workforce and Education, IBM Quantum

IBM **Quantum**

IBM

Qiskit Overview

Qiskit SDK

arXiv:2405.08810

The lingua franca of quantum computing; write once and execute quantum circuits on 10+ different hardware providers

IBM Quantum

AQT

IQM

Azure Quantum

ALICE & BOB

IONQ

Amazon Braket
Get started with quantum computing

Superstaq

QUANTINUUM

rigetti

Quantum SDK Preferred
(2025 Unitary Fund Survey)

69%

Qiskit contributors that are external to IBM

74%

Dependent Qiskit Projects

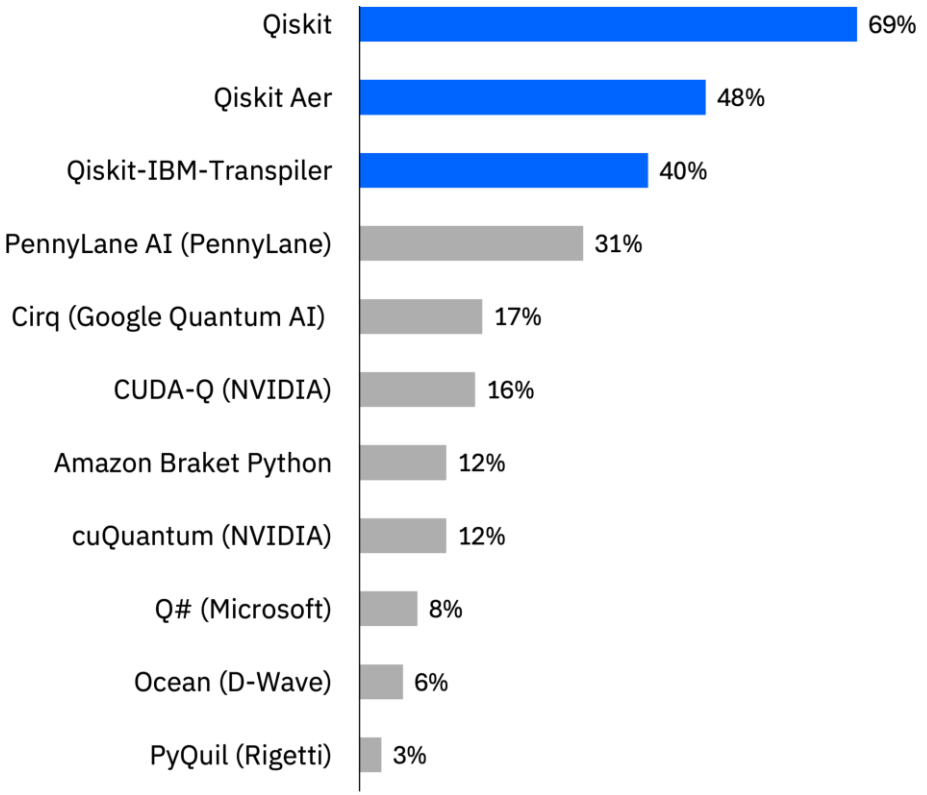
7400+

Qiskit is the most preferred and performant quantum SDK

Speed	129x faster in wall clock time (Qiskit 2.0.2 vs. Tket 2.6.0)	83x faster mean transpilation time (Qiskit 2.2.0 vs. Tket 2.6.0)
Quality	29% fewer 2Q gates (Qiskit 2.2.0 vs. Tket 2.6.0)	Only SDK not to fail any tests
Extensibility	5,700+ more dependent projects (Qiskit: 7400+ vs. PennyLane: 1700+)	

Read the Benchpress paper
↳ https://www.nature.com/articles/s43588-025-00792-y

Full-stack development platforms, compilers, and simulators adoption



Source: Unitary Foundation 2025 Quantum Open Source Software Survey

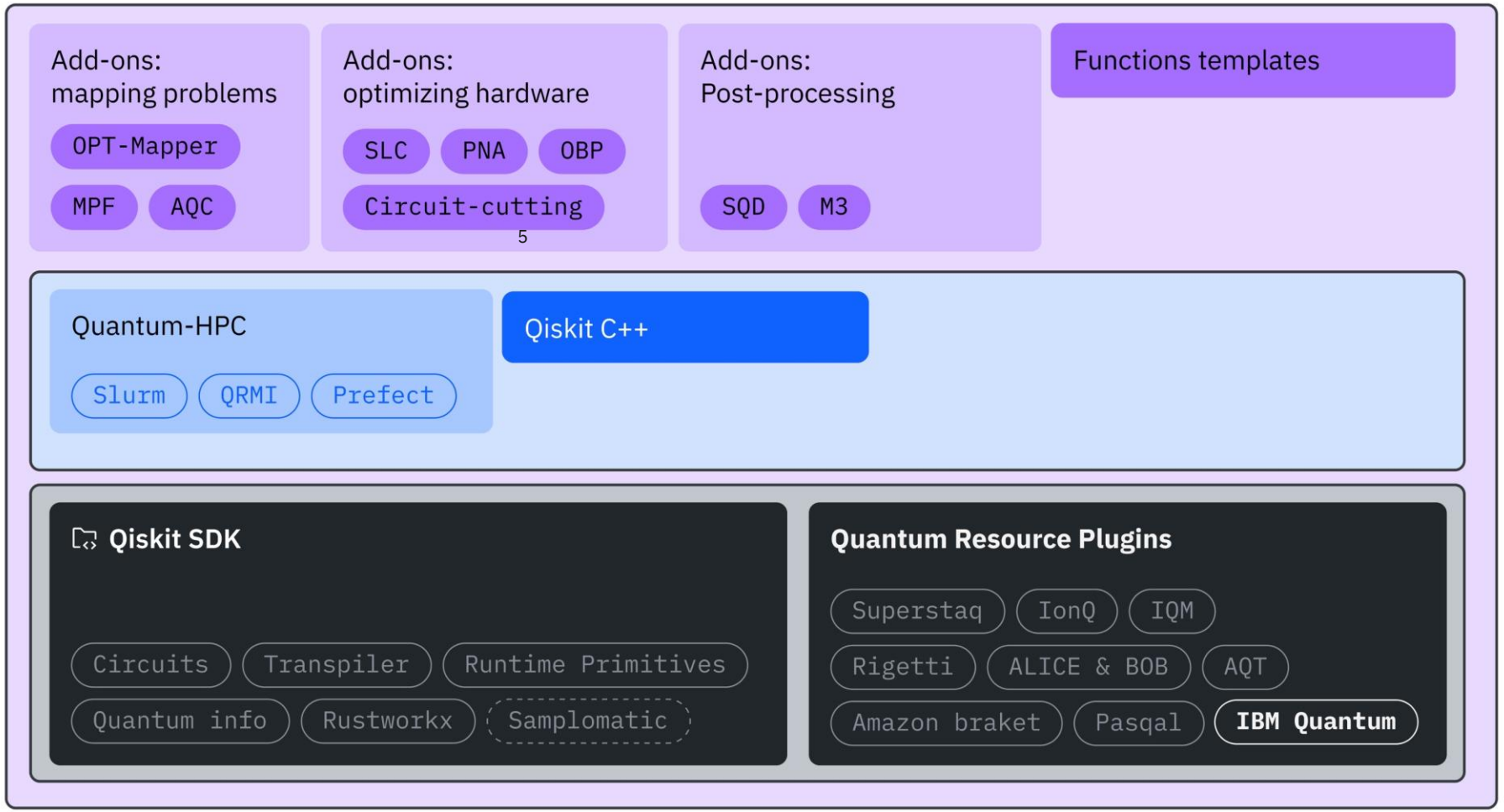
Results reported from Benchpress tests of ~1000 circuits (https://github.com/Qiskit/benchpress) and published in *Nature Computational Science* (2025): https://www.nature.com/articles/s43588-025-00792-y. Timing and quality measured using only completed tests. Dependencies from Github insights. Current as of 12/9/25.

Qiskit –
Open ecosystem for quantum computing

Qiskit for:
- Algorithm research

Qiskit for:
- High Performance Computing

Qiskit for:
- Quantum information science research

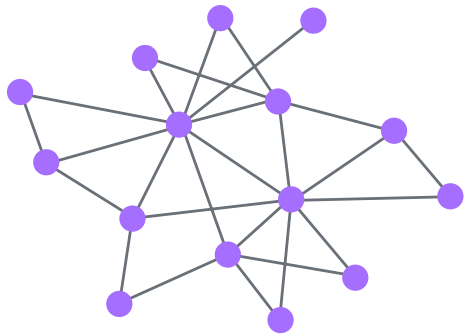


© 2026 IBM Corporation

Qiskit patterns

Step 1

Map classical inputs to a quantum problem



Step 2

Optimize problem for quantum execution.

```
PassManager([UnitarySynthesis(),
             BasisTranslator(),
             EnlargeWithAncilla(),
             AlSwap(),
             Collect1qRuns(),
             Optimize1qGates(),
             Collect2qBlocks(),
             ConsolidateBlocks()])
```

Step 3

Execute using Qiskit Runtime Primitives.

circuit($\vec{\theta}$)

Sampler →

000101...,
110110...

bit-strings

circuit($\vec{\theta}$) + observable $\hat{O}$

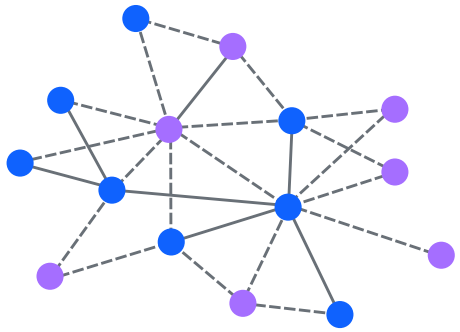
Estimator →

$\langle O \rangle$

expectation value

Step 4

Post-process, return result in classical format.



Qiskit SDK sets the foundation

Qiskit SDK gives us a base layer of building blocks for building and running quantum algorithms

Qiskit Circuit Library

Input:
Domain inputs

Output:
Circuits, observable

Map

Transpiler

Input:
Circuits, observable

Output:
ISA circuit, observable

Optimize

Primitives

Input:
ISA circuit, observable

Output:
Expectation value/samples

Execute

Quantum Info

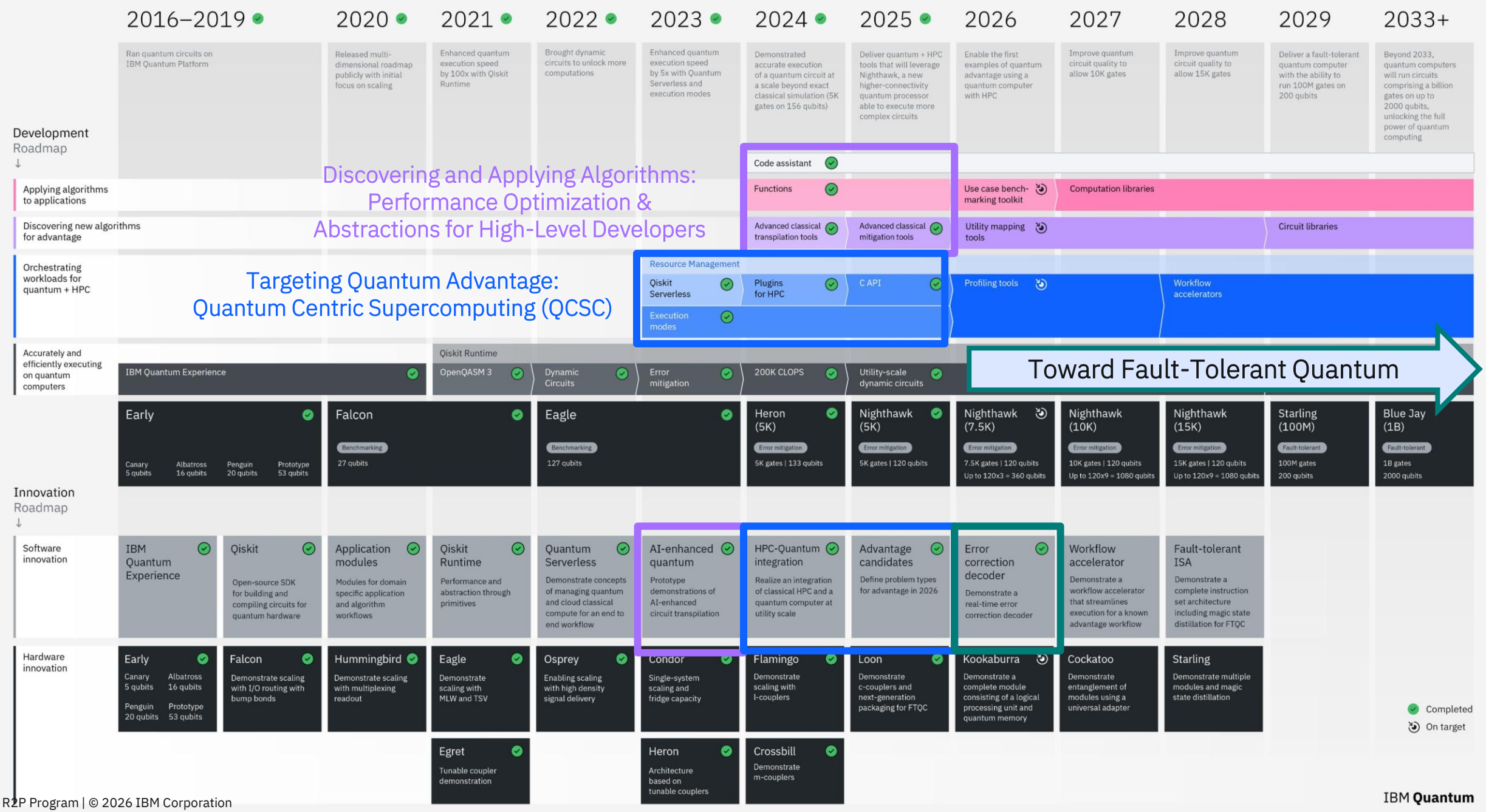
Input:
Expectation value/samples

Output:
Data objects/visualizations

Post-Process

© 2026 IBM Corporation

Qiskit v2 Updates


2016–2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2033+
Ran quantum circuits on IBM Quantum Platform
Released multi-dimensional roadmap publicly with initial focus on scaling
Enhanced quantum execution speed by 100x with Qiskit Runtime
Brought dynamic circuits to unlock more computations
Enhanced quantum execution speed by 5x with Quantum Serverless and execution modes
Demonstrated accurate execution of a quantum circuit at a scale beyond exact classical simulation (5K gates on 156 qubits)
Deliver quantum + HPC tools that will leverage Nighthawk, a new higher-connectivity quantum processor able to execute more complex circuits
Enable the first examples of quantum advantage using a quantum computer with HPC
Improve quantum circuit quality to allow 10K gates
Improve quantum circuit quality to allow 15K gates
Deliver a fault-tolerant quantum computer with the ability to run 100M gates on 200 qubits
Beyond 2033, quantum computers will run circuits comprising a billion gates on up to 2000 qubits, unlocking the full power of quantum computing
Development Roadmap
Discovering and Applying Algorithms: Performance Optimization & Abstractions for High-Level Developers
Targeting Quantum Advantage: Quantum Centric Supercomputing (QCSC)
Toward Fault-Tolerant Quantum
Applying algorithms to applications
Discovering new algorithms for advantage
Orchestrating workloads for quantum + HPC
Accurately and efficiently executing on quantum computers
Code assistant
Functions
Use case bench-marking toolkit
Computation libraries
Advanced classical transpilation tools
Advanced classical mitigation tools
Utility mapping tools
Circuit libraries
Resource Management
Qiskit Serverless
Plugins for HPC
C API
Execution modes
Profiling tools
Workflow accelerators
Qiskit Runtime
IBM Quantum Experience
OpenQASM 3
Dynamic Circuits
Error mitigation
200K CLOPS
Utility-scale dynamic circuits
Early
Canary 5 qubits
Albatross 16 qubits
Penguin 20 qubits
Prototype 53 qubits
Falcon
Benchmarking
27 qubits
Eagle
Benchmarking
127 qubits
Heron (5K)
Error mitigation
5K gates | 133 qubits
Nighthawk (5K)
Error mitigation
5K gates | 120 qubits
Nighthawk (7.5K)
Error mitigation
7.5K gates | 120 qubits
Up to 120x3 = 360 qubits
Nighthawk (10K)
Error mitigation
10K gates | 120 qubits
Up to 120x9 = 1080 qubits
Nighthawk (15K)
Error mitigation
15K gates | 120 qubits
Up to 120x9 = 1080 qubits
Starling (100M)
Fault-tolerant
100M gates
200 qubits
Blue Jay (1B)
Fault-tolerant
1B gates
2000 qubits
Innovation Roadmap
Software innovation
IBM Quantum Experience
Qiskit
Open-source SDK for building and compiling circuits for quantum hardware
Application modules
Modules for domain specific application and algorithm workflows
Qiskit Runtime
Performance and abstraction through primitives
Quantum Serverless
Demonstrate concepts of managing quantum and cloud classical compute for an end to end workflow
AI-enhanced quantum
Prototype demonstrations of AI-enhanced circuit transpilation
HPC-Quantum integration
Realize an integration of classical HPC and a quantum computer at utility scale
Advantage candidates
Define problem types for advantage in 2026
Error correction decoder
Demonstrate a real-time error correction decoder
Workflow accelerator
Demonstrate a workflow accelerator that streamlines execution for a known advantage workflow
Fault-tolerant ISA
Demonstrate a complete instruction set architecture including magic state distillation for FTQC
Hardware innovation
Early
Canary 5 qubits
Albatross 16 qubits
Penguin 20 qubits
Prototype 53 qubits
Falcon
Demonstrate scaling with I/O routing with bump bonds
Hummingbird
Demonstrate scaling with multiplexing readout
Eagle
Demonstrate scaling with MLW and TSV
Osprey
Enabling scaling with high density signal delivery
Condor
Single-system scaling and fridge capacity
Flamingo
Demonstrate scaling with l-couplers
Loon
Demonstrate c-couplers and next-generation packaging for FTQC
Kookaburra
Demonstrate a complete module consisting of a logical processing unit and quantum memory
Cockatoo
Demonstrate entanglement of modules using a universal adapter
Starling
Demonstrate multiple modules and magic state distillation
Egret
Tunable coupler demonstration
Heron
Architecture based on tunable couplers
Crossbill
Demonstrate m-couplers
Completed
On target
IBM Quantum

Key milestones in bringing useful quantum computing to the world

2023

Establish quantum utility

2026

Demonstrate quantum advantage

2029

Deliver the first large-scale, fault-tolerant quantum computer

© 2026 IBM Corporation

Qiskit C API

```c
#define PY_SSIZE_T_CLEAN
#include <Python.h> // include Python header for PyObject
#define QISKIT_C_PYTHON_INTERFACE // enable C->Python conversions
#include <qiskit.h>

PyObject *build_ghz(uint32_t num_bits) {
    QkCircuit *circuit = qk_circuit_new(num_bits, num_bits);

    // Add Hadamard on qubit 0
    uint32_t q0[1] = {0};
    qk_circuit_gate(circuit, QkGate_H, q0, NULL);

    // Add the CX gates
    uint32_t inter[2] = {0, 0};
    for (uint32_t i = 2; i < num_bits; i++) {
        inter[1] = i;
        qk_circuit_gate(circuit, QkGate_CX, inter, NULL);
    }

    // Measure the qubits
    for (uint32_t i = 0; i < num_bits; i++) {
        qk_circuit_measure(circuit, i, i);
    }
    return qk_circuit_to_python(circuit);
}
```

```python
def build_ghz_fast(n: int) -> QuantumCircuit:
  """Build a GHZ state skipping checks where possible."""
  circuit = QuantumCircuit(n, n)
  # use the internal Rust classes directly
  h = clib.HGate._standard_gate
  cx = clib.CXGate._standard_gate
  circuit._append(
  CircuitInstruction.from_standard(
    h, [circuit.qubits[0]], [])
  )
  for i in range(1, n):
    circuit._append(
    CircuitInstruction.from_standard(
      cx, [circuit.qubits[0], circuit.qubits[i]], [])
    )
  for i in range(n):
    circuit._append(
    CircuitInstruction(
      clib.Measure(),
      [circuit.qubits[i]], [circuit.clbits[i]])
    )
  return circuit
```

Time to create a 1000 qubit GHZ state Circuit:

- **C and Python:** 342 µs ± 100 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
- **Python:** 4.64 ms ± 330 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)

Other Language Bindings to Qiskit C API

- C++ https://github.com/Qiskit/qiskit-cpp

```cpp
#include "circuit/quantumcircuit.hpp"
using namespace Qiskit::circuit;
int main() {
 QuantumRegister qr(2);
 ClassicalRegister cr(2);
 QuantumCircuit circ(qr, cr);
 circ.h(0);
 circ.cx(0, 1);
 circ.measure(0, 0);
 circ.measure(1, 1);
 return 0;
}
```

Other Language Bindings to Qiskit C API

- C++ https://github.com/Qiskit/qiskit-cpp
- Rust https://github.com/Qiskit/qiskit-rs

```rust
use qiskit_rs::QuantumCircuit;
pub fn main() {
 let mut qc = QuantumCircuit::new(2, 2);
 qc.h(0);
 qc.cx(0, 1);
 qc.measure(0, 0);
 qc.measure(1, 1);
}
```

Other Language Bindings to Qiskit C API

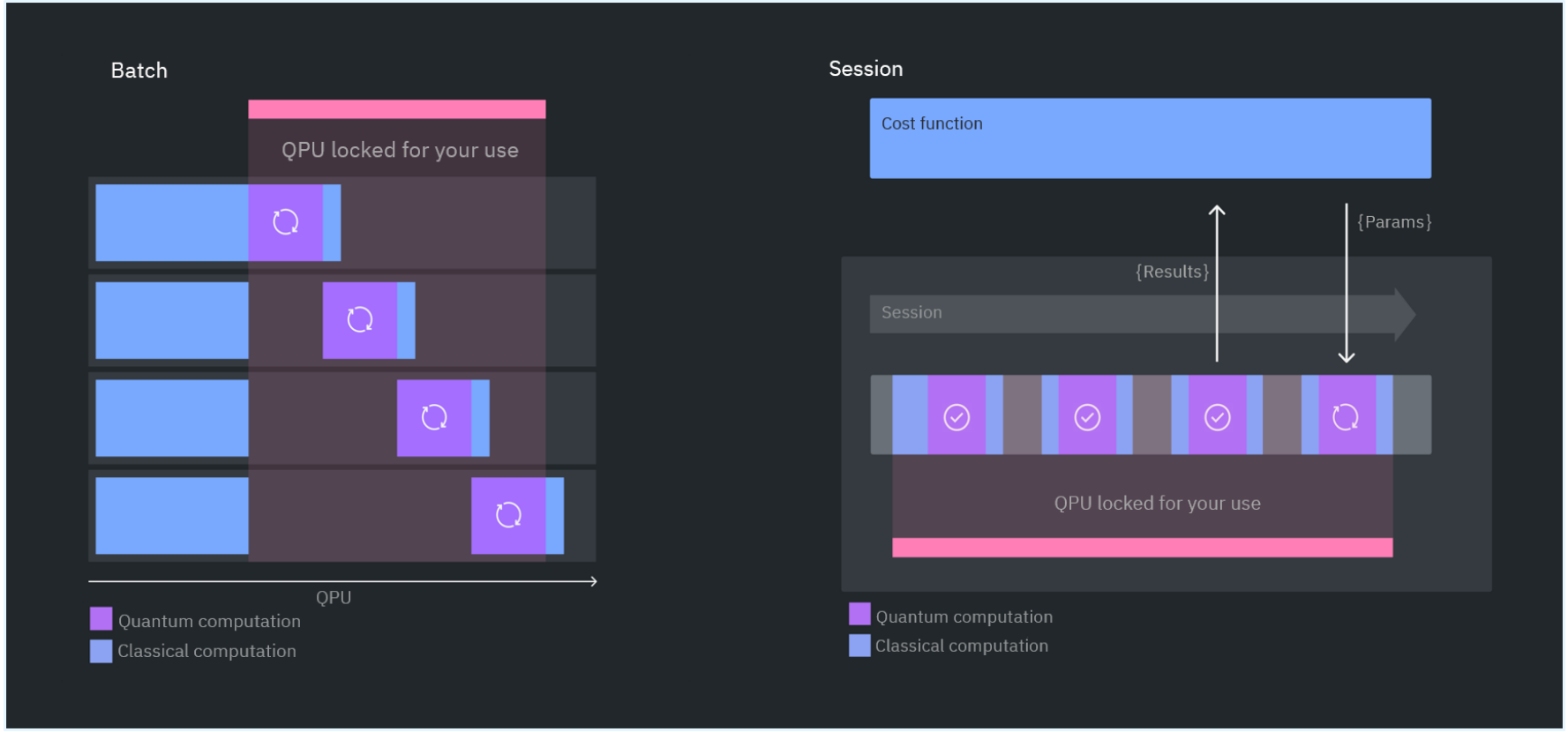
- C++ https://github.com/Qiskit/qiskit-cpp
- Rust https://github.com/Qiskit/qiskit-rs
- Julia https://github.com/Qiskit/Qiskit.jl/

```julia
using Qiskit
function build_bell()
 qc = Qiskit.Circuit(2, 2)
 qc.h(1)
 qc.cx(1, 2)
 qc.measure(1, 1)
 qc.measure(2, 2)
 qc
end
```

Execution Modes

```python
# Batch mode: useful for a collection of independent jobs
with Batch(backend=backend, max_time="2m") as batch:
    sampler_batch = Sampler(mode=batch)
    estimator_batch = Estimator(mode=batch)
```

```python
# Session mode: useful for iterative workloads,
# open-access users cannot use this mode.
with Session(backend=backend, max_time="2m") as session:
  sampler_session = Sampler(mode=session)
  estimator_session = Estimator(mode=session)
```



Qiskit Code Assistant:
AI-Powered Quantum Development

Purpose:

- Qiskit Code Assistant (QCA) enhances quantum computing accessibility and improves developer productivity through generative AI-powered code suggestions.

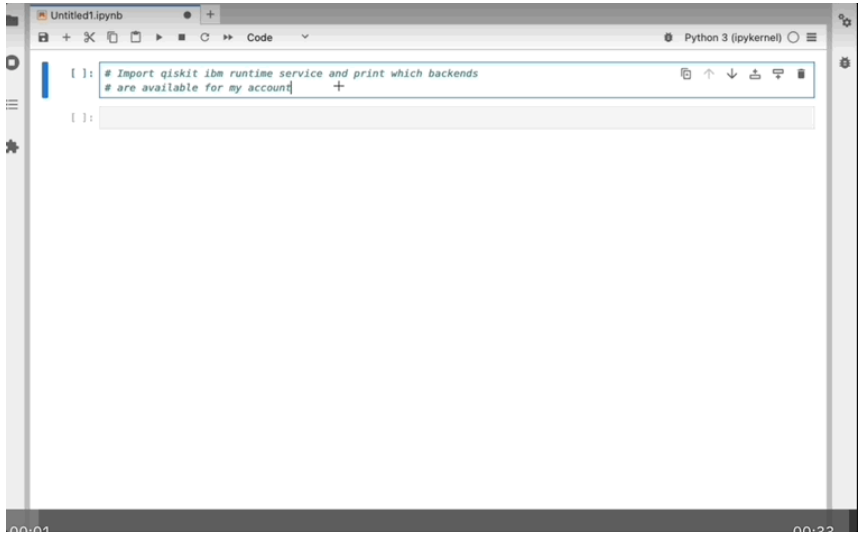
Core Capabilities:

- Generates context-aware Qiskit code from natural language prompts or partial code
- Trained on millions of tokens from Qiskit SDK v1.x, GitHub repositories, documentation, and tutorials
- Powered by IBM Granite 3.3-8B models specialized for quantum computing (via watsonx)
- Supports both abstract queries and specific code completion requests

Technical Features:

- Available as extensions for VS Code and JupyterLab
- Supports Python files and Jupyter notebooks
- Real-time code suggestions with review, accept, or reject workflow
- Built-in guardrails for responsible AI usage

+



Access here: quantum.cloud.ibm.com/docs/en/guides/qiskit-code-assistant

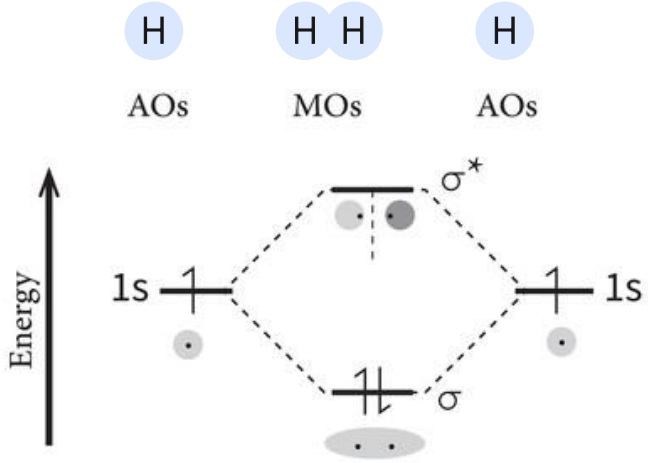
Qiskit Developer Certification

The Qiskit Developer Certification is the world's first ever developer certification for programming a quantum computer. There are over 1,100+ certified Qiskit developers across the globe. This certificate sets a benchmark for the industry in identifying those with quantum developer skills.

VQE Overview

Quantum Chemistry for Electronic Structures

Single configuration ground state:



H_2 Molecule Orbitals
(minimal basis; sto-3g)

$E_{HF} = -1.11$ Ha (inaccurate)

Full Configuration Interaction:

$$\left|\Psi_{\mathrm{FCI}}^{\mathrm{H}_2}\right\rangle = \sum_{I=1}^{6} c_I \left|D_I\right\rangle$$

$$c_1 \begin{bmatrix} \uparrow\downarrow & \\ \sigma_g & \sigma_u^* \\ \downarrow & \uparrow \\ \sigma_g & \sigma_u^* \end{bmatrix} + c_2 \begin{bmatrix} \uparrow & \uparrow \\ \sigma_g & \sigma_u^* \\ \downarrow & \downarrow \\ \sigma_g & \sigma_u^* \end{bmatrix} + c_3 \begin{bmatrix} \uparrow & \downarrow \\ \sigma_g & \sigma_u^* \\ & \uparrow\downarrow \\ \sigma_g & \sigma_u^* \end{bmatrix}$$

$$c_4 \qquad\qquad + c_5 \qquad\qquad + c_6$$

$$\text{Minimal-basis } H_2: \quad \binom{4}{2} = 6 \quad \text{determinants from} \quad \{\sigma_g\uparrow, \sigma_g\downarrow, \sigma_u^*\uparrow, \sigma_u^*\downarrow\}.$$

2 electrons in 4 spin orbitals
Determine the coefficients $c_1 \ldots c_6$

$E_{FCI} = -1.13$ Ha (accurate)

Configurations increases combinatorically with size (Example: Benzene C_6H_6)

C (1s, 2s, 2p):	10 AO, 6 E
H (1s):	2 AO, 2 E
C_6H_6:	72 MO, 42 E

$$\binom{72}{42} \approx 4.7 \times 10^{17}$$

© 2026 IBM Corporation

Ground State Energy - Classical

$$H = \begin{bmatrix} H_{00} & \cdots & H_{0N} \\ \vdots & \ddots & \vdots \\ H_{N0} & \cdots & H_{NN} \end{bmatrix} \rightarrow \text{Eigenvalue problem } Hc = Ec \rightarrow \text{Eigenvalue problem } E_{gnd} = \min(E)$$

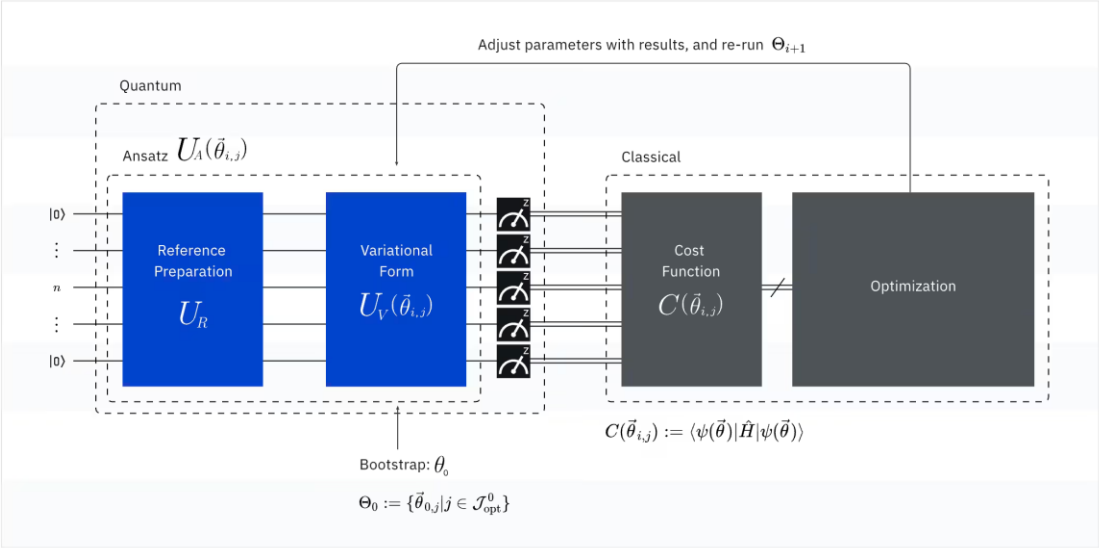
Exact Classical Method (Full Configuration Interaction):

1. Construct **Hamiltonian matrix** with size of # configurations (N).
2. Perform diagonalization (i.e. eigenvalue problem) $T \sim O(N^3)$
3. Find the minimal eigenvalue.

(Not a scalable method)

© 2026 IBM Corporation

Variational Quantum Eigensolver (VQE)



1. Prepare the parameterized quantum state

$$|\psi(\boldsymbol{\theta})\rangle = U(\boldsymbol{\theta})|\psi_{\text{ref}}\rangle$$

2. Measure the expectation value

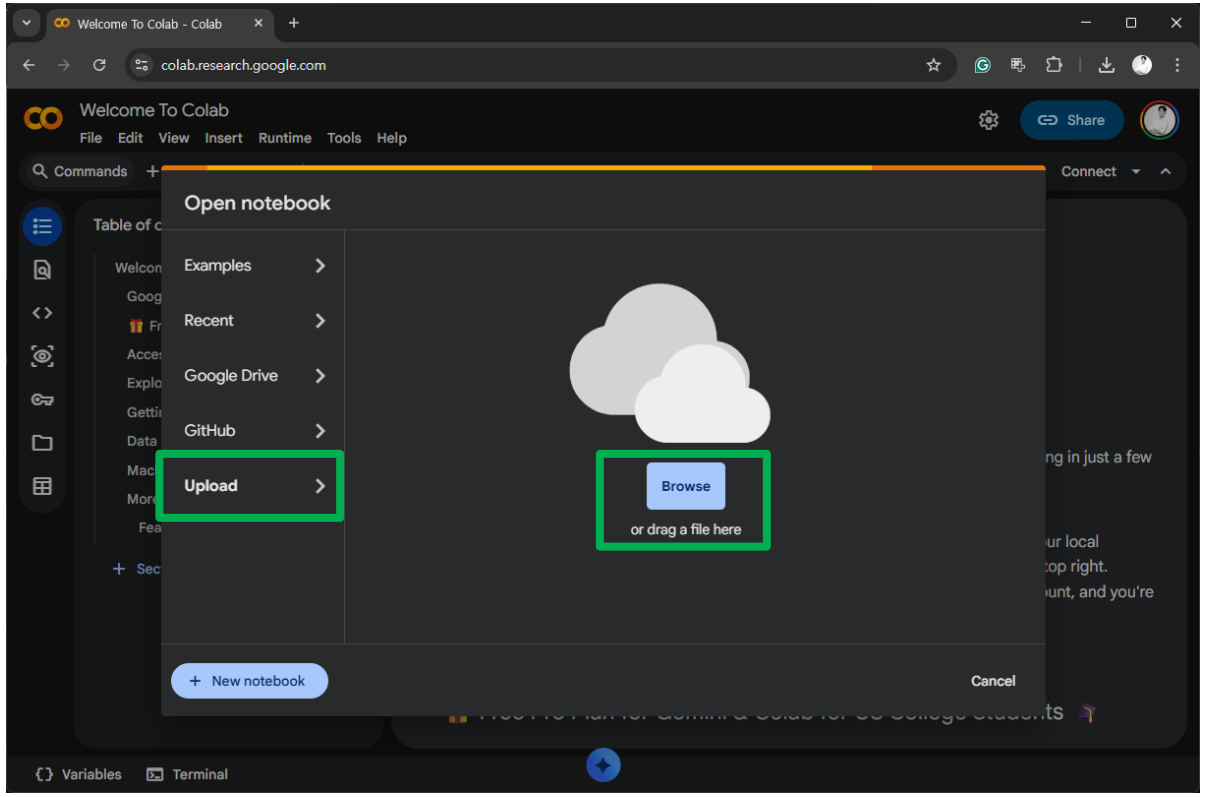
$$E(\boldsymbol{\theta}) = \langle\psi(\boldsymbol{\theta})|H|\psi(\boldsymbol{\theta})\rangle$$

3. Classically update the parameter to obtain the lowest energy

$$E_{VQE} = \min_{\boldsymbol{\theta}} E(\boldsymbol{\theta})$$

© 2026 IBM Corporation

Now, let’s move on to the **hands-on session!**



1. Go to https://colab.research.google.com

2. Upload the provided ipynb file.
   If not provided, download:
   https://github.com/snow0369/qiskit_handson_260330
   “combined_colab.ipynb”

3. Log-in IBM Quantum Platform:
   https://quantum.cloud.ibm.com/

© 2026 IBM Corporation